

Phase 1: Conceptual Understanding

Kimon Anagnostopoulos

January 2026

Abstract

This document describes the design and implementation of an interactive Boids flocking simulation. The system extends Craig Reynolds' classic algorithm (1986) with predator-prey dynamics, obstacle avoidance, and real-time parameter tuning. The implementation features a Python/FastAPI backend with KD-Tree optimization achieving $O(n \log n)$ neighbor queries, streaming frames at 60 FPS via WebSocket to a React frontend. Multiple predator hunting strategies create emergent defensive behaviors in the flock.

Contents

1	Introduction	3
1.1	Background	3
2	Mathematical Model	3
3	Flocking Rules	3
3.1	Separation	3
3.2	Alignment	4
3.3	Cohesion	4
4	Boundary and Speed Constraints	5
4.1	Boundary Steering	5
4.2	Speed Limits	5
5	Predator-Prey Dynamics	6
5.1	Predator Avoidance	6
5.2	Hunting Strategies	6
5.2.1	Hawk Strategy (Center Targeting)	6
5.2.2	Falcon Strategy (Nearest Pursuit)	6
5.2.3	Eagle Strategy (Straggler Hunting)	7
5.2.4	Kite Strategy (Patrol)	7
5.2.5	Osprey Strategy (Random Switching)	7
6	Obstacle Avoidance	7
7	Velocity Update Equation	7
8	Computational Optimization	7
9	Parameter Configuration	8
10	Emergent Behaviors	8

11 System Architecture	8
11.1 Overview	8
11.2 Communication Protocol	9
A Complete Simulation Loop	9

1 Introduction

1.1 Background

Boids is an artificial life program developed by Craig Reynolds in 1986 that produces realistic simulations of the flocking behavior of birds. The fundamental insight of the Boids model is that complex, realistic flocking behavior can emerge from simple local rules applied to each individual agent. Each boid follows three simple steering behaviors based only on the positions and velocities of nearby flockmates:

1. **Separation:** Steer to avoid crowding nearby flockmates
2. **Alignment:** Steer towards the average heading of nearby flockmates
3. **Cohesion:** Steer towards the average position (center of mass) of nearby flockmates

This implementation extends the basic Boids algorithm with multiple predators with distinct hunting strategies, static obstacle avoidance, KD-Tree spatial optimization, interactive parameter tuning via WebSocket streaming, and comprehensive analysis tools.

2 Mathematical Model

Each boid b_i is represented by its position and velocity vectors:

$$\mathbf{p}_i = (x_i, y_i) \in \mathbb{R}^2 \quad (1)$$

$$\mathbf{v}_i = (v_{x,i}, v_{y,i}) \in \mathbb{R}^2 \quad (2)$$

The complete state of a flock with n boids is:

$$S = \{(\mathbf{p}_1, \mathbf{v}_1), (\mathbf{p}_2, \mathbf{v}_2), \dots, (\mathbf{p}_n, \mathbf{v}_n)\} \quad (3)$$

Boids have limited perception defined by two radii:

- **Visual range** r_v : Maximum distance at which a boid can perceive neighbors
- **Protected range** r_p : Minimum comfortable distance ($r_p < r_v$)

The set of neighbors for boid i within range r is:

$$N_i(r) = \{j : j \neq i \wedge \|\mathbf{p}_j - \mathbf{p}_i\| < r\} \quad (4)$$

3 Flocking Rules

3.1 Separation

Separation prevents boids from colliding by steering away from neighbors within the protected range. The separation force is computed as:

$$\mathbf{F}_{\text{sep},i} = \alpha_s \sum_{j \in N_i(r_p)} (\mathbf{p}_i - \mathbf{p}_j) \quad (5)$$

where α_s is the separation strength parameter.

Algorithm 1: Separation Rule

Input: Boid b , neighbors N , protected range r_p , strength α_s
Output: Velocity adjustment $\Delta \mathbf{v}$
 $\mathbf{F} \leftarrow (0, 0)$
foreach neighbor $n \in N$ **do**
 $d \leftarrow \|\mathbf{p}_b - \mathbf{p}_n\|$
 if $d < r_p$ **then**
 $\mathbf{F} \leftarrow \mathbf{F} + (\mathbf{p}_b - \mathbf{p}_n)$
 end
end
return $\alpha_s \cdot \mathbf{F}$

3.2 Alignment

Alignment causes boids to match the velocity of nearby flockmates:

$$\mathbf{F}_{\text{ali},i} = \alpha_a \left(\frac{1}{|N_i(r_v)|} \sum_{j \in N_i(r_v)} \mathbf{v}_j - \mathbf{v}_i \right) \quad (6)$$

where α_a is the alignment factor.

Algorithm 2: Alignment Rule

Input: Boid b , neighbors N , visual range r_v , factor α_a
Output: Velocity adjustment $\Delta \mathbf{v}$
 $\mathbf{V}_{\text{sum}} \leftarrow (0, 0)$
 $\text{count} \leftarrow 0$
foreach neighbor $n \in N$ **do**
 $d \leftarrow \|\mathbf{p}_b - \mathbf{p}_n\|$
 if $d < r_v$ **then**
 $\mathbf{V}_{\text{sum}} \leftarrow \mathbf{V}_{\text{sum}} + \mathbf{v}_n$
 $\text{count} \leftarrow \text{count} + 1$
 end
end
if $\text{count} > 0$ **then**
 $\mathbf{V}_{\text{avg}} \leftarrow \mathbf{V}_{\text{sum}} / \text{count}$
 return $\alpha_a \cdot (\mathbf{V}_{\text{avg}} - \mathbf{v}_b)$
end
return $(0, 0)$

3.3 Cohesion

Cohesion steers boids toward the center of mass of nearby flockmates:

$$\mathbf{F}_{\text{coh},i} = \alpha_c \left(\frac{1}{|N_i(r_v)|} \sum_{j \in N_i(r_v)} \mathbf{p}_j - \mathbf{p}_i \right) \quad (7)$$

where α_c is the cohesion factor.

Algorithm 3: Cohesion Rule

Input: Boid b , neighbors N , visual range r_v , factor α_c
Output: Velocity adjustment $\Delta \mathbf{v}$
 $\mathbf{P}_{\text{sum}} \leftarrow (0, 0)$
 $\text{count} \leftarrow 0$
foreach neighbor $n \in N$ **do**
 $d \leftarrow \|\mathbf{p}_b - \mathbf{p}_n\|$
 if $d < r_v$ **then**
 $\mathbf{P}_{\text{sum}} \leftarrow \mathbf{P}_{\text{sum}} + \mathbf{p}_n$
 $\text{count} \leftarrow \text{count} + 1$
 end
end
if $\text{count} > 0$ **then**
 $\mathbf{P}_{\text{avg}} \leftarrow \mathbf{P}_{\text{sum}} / \text{count}$
 return $\alpha_c \cdot (\mathbf{P}_{\text{avg}} - \mathbf{p}_b)$
end
return $(0, 0)$

4 Boundary and Speed Constraints

4.1 Boundary Steering

Boids must turn naturally when approaching screen edges. We apply a gradual turning force when within the margin zone instead of hard boundaries as follows:

$$F_{\text{bnd},x} = \begin{cases} +\tau & \text{if } x < m \\ -\tau & \text{if } x > W - m \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

where m is the margin distance, W is the screen width, and τ is the turn factor. The same logic applies to the y -component with screen height H .

Algorithm 4: Boundary Steering

Input: Boid b , margins (m_L, m_R, m_T, m_B) , turn factor τ
Output: Velocity adjustment $\Delta \mathbf{v}$
 $\Delta v_x \leftarrow 0$
 $\Delta v_y \leftarrow 0$
if $b.x < m_L$ **then** $\Delta v_x \leftarrow +\tau$
else if $b.x > m_R$ **then** $\Delta v_x \leftarrow -\tau$
if $b.y < m_T$ **then** $\Delta v_y \leftarrow +\tau$
else if $b.y > m_B$ **then** $\Delta v_y \leftarrow -\tau$
return $(\Delta v_x, \Delta v_y)$

4.2 Speed Limits

Boid speed is constrained to the range $[v_{\min}, v_{\max}]$:

$$\mathbf{v}' = \begin{cases} \frac{\mathbf{v}}{\|\mathbf{v}\|} \cdot v_{\max} & \text{if } \|\mathbf{v}\| > v_{\max} \\ \frac{\mathbf{v}}{\|\mathbf{v}\|} \cdot v_{\min} & \text{if } \|\mathbf{v}\| < v_{\min} \wedge \|\mathbf{v}\| > 0 \\ \mathbf{v} & \text{otherwise} \end{cases} \quad (9)$$

5 Predator-Prey Dynamics

5.1 Predator Avoidance

When a predator is present, boids within the predator's detection range experience an additional avoidance force:

$$\mathbf{F}_{\text{avoid},i} = \alpha_p \cdot \frac{\mathbf{p}_i - \mathbf{p}_{\text{pred}}}{\|\mathbf{p}_i - \mathbf{p}_{\text{pred}}\|^2} \quad (10)$$

where α_p is the predator avoidance strength. The inverse-square relationship creates stronger avoidance at closer distances.

Algorithm 5: Predator Avoidance

Input: Boid b , predator p , detection range r_d , strength α_p

Output: Velocity adjustment $\Delta \mathbf{v}$

$\mathbf{d} \leftarrow \mathbf{p}_b - \mathbf{p}_p$

$dist \leftarrow \|\mathbf{d}\|$

if $dist < r_d$ **and** $dist > 0$ **then**

return $\alpha_p \cdot \mathbf{d}/dist^2$

end

return $(0, 0)$

5.2 Hunting Strategies

The implementation supports five predator hunting strategies, each producing different emergent behaviors:

Strategy	Name	Behavior
0	Hawk	Targets the flock's center of mass, causing the flock to split
1	Falcon	Pursues the nearest boid, creating localized panic
2	Eagle	Hunts stragglers at flock edges, picking off isolated boids
3	Kite	Patrols in circular patterns, attacking opportunistically
4	Osprey	Randomly switches targets, creating unpredictable pressure

Table 1: Predator hunting strategies and their behavioral effects.

5.2.1 Hawk Strategy (Center Targeting)

The Hawk computes the flock centroid and steers toward it:

$$\mathbf{t}_{\text{hawk}} = \frac{1}{n} \sum_{i=1}^n \mathbf{p}_i \quad (11)$$

5.2.2 Falcon Strategy (Nearest Pursuit)

The Falcon targets the closest boid:

$$\mathbf{t}_{\text{falcon}} = \mathbf{p}_j \text{ where } j = \arg \min_i \|\mathbf{p}_i - \mathbf{p}_{\text{pred}}\| \quad (12)$$

5.2.3 Eagle Strategy (Straggler Hunting)

The Eagle identifies boids far from the flock center and near edges:

$$\text{score}(b_i) = \|\mathbf{p}_i - \mathbf{c}\| + \lambda \cdot \text{edge_distance}(b_i) \quad (13)$$

where $\mathbf{c}$ is the flock centroid and λ weights the edge proximity.

5.2.4 Kite Strategy (Patrol)

The Kite follows a circular patrol path:

$$x_{\text{patrol}}(t) = c_x + R \cos(\omega t) \quad (14)$$

$$y_{\text{patrol}}(t) = c_y + R \sin(\omega t) \quad (15)$$

and attacks any boid entering its patrol radius.

5.2.5 Osprey Strategy (Random Switching)

The Osprey selects random targets, switching every T frames:

$$\mathbf{t}_{\text{osprey}}(t) = \mathbf{p}_k \text{ where } k \sim \text{Uniform}(1, n), \quad t \bmod T = 0 \quad (16)$$

6 Obstacle Avoidance

Static circular obstacles are defined by position and radius: $O = (\mathbf{p}_O, r_O)$. The avoidance force steers boids away from obstacles within detection range:

$$\mathbf{F}_{\text{obs},i} = \alpha_o \sum_{O \in \mathcal{O}} \begin{cases} \frac{\mathbf{p}_i - \mathbf{p}_O}{\|\mathbf{p}_i - \mathbf{p}_O\| - r_O} & \text{if } \|\mathbf{p}_i - \mathbf{p}_O\| < r_O + r_d \\ \mathbf{0} & \text{otherwise} \end{cases} \quad (17)$$

where r_d is the obstacle detection range and α_o is the avoidance strength.

7 Velocity Update Equation

The complete velocity update for boid i combines all forces:

$$\mathbf{v}_i^{(t+1)} = \text{clamp} \left(\mathbf{v}_i^{(t)} + \mathbf{F}_{\text{sep}} + \mathbf{F}_{\text{ali}} + \mathbf{F}_{\text{coh}} + \mathbf{F}_{\text{bnd}} + \mathbf{F}_{\text{avoid}} + \mathbf{F}_{\text{obs}}, v_{\min}, v_{\max} \right) \quad (18)$$

Position is then updated:

$$\mathbf{p}_i^{(t+1)} = \mathbf{p}_i^{(t)} + \mathbf{v}_i^{(t+1)} \quad (19)$$

8 Computational Optimization

The naive implementation requires checking all pairs of boids for neighbor queries, resulting in $O(n^2)$ complexity per frame. With $n = 200$ boids at 60 FPS, this becomes computationally prohibitive. We employ a KD-Tree spatial data structure to accelerate neighbor queries:

- **Construction:** $O(n \log n)$ per frame (tree must be rebuilt as boids move)
- **Range query:** $O(\sqrt{n} + k)$ where k is the number of neighbors found
- **Total complexity:** $O(n \log n + n\sqrt{n})$ vs $O(n^2)$ naive

The KD-Tree is implemented using SciPy's `scipy.spatial.KDTree` with the `query_ball_point` method for range queries.

Boids	Naive (ms)	KD-Tree (ms)	Speedup
50	2.1	0.9	2.4×
100	7.8	1.6	4.9×
200	30.2	3.1	9.7×
500	185.4	7.8	23.8×

Table 2: Performance comparison: Naive vs KD-Tree optimized implementation.

9 Parameter Configuration

Parameter	Symbol	Range	Default
<i>Boid Behavior</i>			
Number of boids	n	1–200	50
Visual range	r_v	10–150	50
Protected range	r_p	5–50	20
Separation strength	α_s	0.01–0.5	0.15
Alignment factor	α_a	0.01–0.2	0.06
Cohesion factor	α_c	0.0001–0.02	0.002
<i>Speed Constraints</i>			
Maximum speed	$v_{\max}$	1–8	6.0
Minimum speed	$v_{\min}$	0–3	0.0
<i>Boundary</i>			
Margin	m	25–150	75
Turn factor	τ	0.1–1.0	0.2
<i>Predator</i>			
Predator speed	v_p	0.5–5	2.5
Detection range	r_d	50–300	150
Avoidance strength	α_p	0.05–1.5	0.8
Number of predators	—	1–5	1

Table 3: Complete parameter specification with valid ranges.

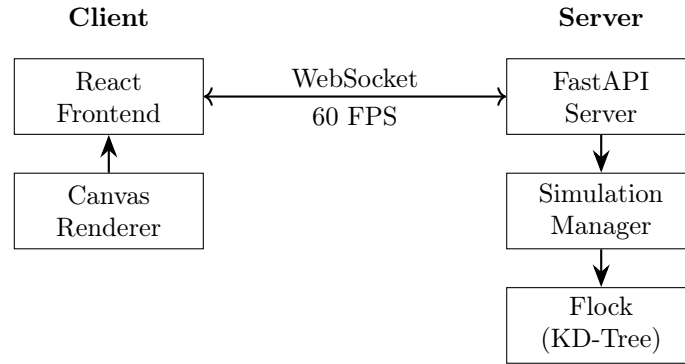
10 Emergent Behaviors

From the local rules, the several complex behaviors emerge. Initially random boids form coherent groups. Directional changes propagate through these groups as they turn collectively. If an obstacle is present, flocks are expected to fragment around them and then reform. In addition, spacing between inter-boid distances should eventually stabilize. When a predator is present, approaching the flock should trigger flash expansion. Predator induced behavior is expected to vary across predator species as specified in Section 5.2.

11 System Architecture

11.1 Overview

The implementation uses a client-server architecture with WebSocket communication:



11.2 Communication Protocol

Server → Client (60 FPS):

```
{
  "type": "frame",
  "frame_id": 1234,
  "boids": [[x, y, vx, vy], ...],
  "predators": [{"x": 400, "y": 300, "strategy_name": "Hawk"}],
  "metrics": {"fps": 60.0}
}
```

Client → Server:

```
{"type": "update_params", "params": {"num_boids": 100}}
{"type": "preset", "name": "predator_chase"}
{"type": "pause"} / {"type": "resume"} / {"type": "reset"}
```

References

1. Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics*, 21(4), 25–34.
2. <https://www.red3d.com/cwr/boids/>
3. <https://people.ece.cornell.edu/land/courses/ece4760/labs/s2021/Boids/Boids.html>
4. https://vanhunteradams.com/Pico/Animal_Movement/Boids-predator.html

A Complete Simulation Loop

```
for each frame:
    # Rebuild KD-Tree for efficient neighbor queries
    kdtree = KDTree(boid_positions)

    # Compute forces for all boids (read old state)
    for each boid b:
        neighbors = kdtree.query_ball_point(b.position, visual_range)

        F_sep = compute_separation(b, neighbors, protected_range, sep_strength)
        F_al_i = compute_alignment(b, neighbors, alignment_factor)
        F_coh = compute_cohesion(b, neighbors, cohesion_factor)
        F_bnd = compute_boundary_steering(b, margin, turn_factor)
```

```
F_pred = compute_predator_avoidance(b, predators, detection_range)
F_obs = compute_obstacle_avoidance(b, obstacles)

b.dv = F_sep + F_ali + F_coh + F_bnd + F_pred + F_obs

# Apply forces and update positions
for each boid b:
    b.velocity += b.dv
    b.velocity = clamp_speed(b.velocity, min_speed, max_speed)
    b.position += b.velocity

# Update predators
for each predator p:
    target = p.select_target(boids) # Strategy-dependent
    p.steer_toward(target)
    p.velocity = clamp_speed(p.velocity, predator_speed)
    p.position += p.velocity

# Send frame to client
websocket.send(serialize_frame(boids, predators, metrics))
```